

D.I. Recruitment Web Application

From the first setup to daily administration,
deployment, and a PostgreSQL migration.

Project	D.I. Recruitment Agency website
Stack	Next.js 15, React 19, TypeScript, Prisma 6 and MySQL
Local address	http://localhost:3001
Prepared	23 September 2026
Audience	Website owner, administrator and maintaining developer
Scope	Implemented project, operating procedures and future database migration

How to use this handbook

Read pages 3-7 to understand and run the app. Use pages 8-16 for content management. Pages 17-22 explain implementation, security, testing and deployment. The final section gives a staged MySQL-to-PostgreSQL migration plan.

Important: this PDF documents the current source code and the development work discussed in this project. It is not an independent security certification. It does not contain real credentials, private applications, or contact messages. PostgreSQL migration is guidance only; the running app has not been switched.

Contents & reading map

Pages	Topic
3-4	What was built; architecture and request flows
5-7	Source code map; local setup; seed and everyday commands
8-10	Public navigation, homepage carousel, jobs and applications
11-14	Blog, About, team, legal documents, gallery, Facebook and contact
15-16	Testimonials and the featured opportunity card
17-19	Authentication, upload handling and database model reference
20-22	Testing, troubleshooting, deployment and maintenance
23	References and verification boundaries
24-29	PostgreSQL: preparation, schema conversion, data transfer, sequence repair, cutover and rollback

Terminology

Public dashboard: the visitor-facing homepage, not the administrator console. **CRUD:** create, read, update and delete. **Published:** visible to visitors. **Draft:** saved in the database but normally hidden from public pages. **Prisma Client:** generated TypeScript/JavaScript code used to query your database.

Schema: the model definitions in prisma/schema.prisma. **Seed:** a setup script that inserts initial records. **Migration:** a versioned database structure change; moving existing data between different database engines also needs a separate transfer process.

Commands and paths

Commands are intended to run from the project root. File names throughout this guide are relative to that root unless stated otherwise. Examples use placeholders instead of passwords. Do not copy placeholder values unchanged into production.

D:\PROJECT FOLDER\e-commerce\di recruitment webpage

01 / What was built

The project evolved from a visually styled recruitment homepage into a database-backed content management application. The public website presents recruitment information; the authenticated admin area manages content and reviews visitor submissions.

1. Foundation

A Next.js App Router project uses React components, TypeScript types, CSS styling, Prisma models and a MySQL connection. Server-rendered pages read published records from the database.

2. Recruitment workflows

Job management and application submission were connected to MySQL. The homepage now previews at most four jobs. A separate /jobs page lists all published jobs and supports client-side search, location filters and applications.

3. Content management

Blog posts, ordered team members, About pages, leadership portraits and legal document images became editable in admin. Public pages read saved records rather than requiring code edits for each update.

4. Media and engagement

Event folders support photos/videos. A Facebook Page embed appears on both public and admin dashboards. Homepage slides and testimonials rotate every five seconds when multiple published items exist.

5. Owner controls

Contact enquiries are collected on a separate page. Admin can review/delete them. A dedicated featured opportunity editor controls the hero card. Password changes and strict upload limits were added.

6. Refinements

The public navbar stays at the top during scrolling. Draft/published carousel status is now explicit. A stale Prisma Client that lacked newly added models was regenerated, and the server was restarted.

What remains fixed in code

Not every word is CMS-managed. Some homepage introductory text, service copy, statistics, contact information and styling remain in app/page.tsx and CSS. Verify all public business claims before deployment. There is no full drag-and-drop page builder.

02 / Architecture & data flow

Layer	Responsibility
Browser / React	Renders cards and forms; filters jobs; previews files; runs carousel timers.
Next.js server	Renders database-backed pages, processes server actions and API requests.
Validation / authorization	Zod checks inputs. requireAdmin or session checks protect administrative writes.
Prisma Client	Provides typed database access, generated from the schema.
MySQL	Stores content, users, settings, file references and submissions.
Persistent disk	Stores uploaded media under storage/gallery, storage/team and storage/content.

Admin content update, step by step

1. Admin signs in and opens an editor. 2. The browser submits FormData to a server action. 3. The server verifies the session and validates fields. 4. It validates and stores any new image. 5. Prisma inserts or updates a record. 6. The action revalidates affected routes. 7. A fresh public page request reads the updated published data.

Visitor submission, step by step

The job application modal posts JSON to /api/applications. The server validates and saves an Application linked to a Job. The contact page instead calls a server action and saves a ContactSubmission. Both workflows remain separate so a general enquiry does not need a job ID.

Why files are not all in public/

Uploaded draft images must not be publicly accessible just because their URL is known. Dedicated media routes look up database references and publication state before serving stored files. Static design assets can remain in public/images.

03 / Source code map

Location	Purpose
app/page.tsx	Homepage composition, published jobs and carousel loading.
app/jobs; app/blog	All-jobs page, blog listing and individual articles.
app/about/[slug]; app/team	About pages, legal documents and team presentation.
app/contact; app/gallery	Dedicated enquiry form and event albums.
app/admin	Protected editors, overview, applications, contacts and settings.
app/api	Applications, jobs and authenticated/visibility-aware media endpoints.
components	Reusable forms, navbar, carousels, cards, modals and embedded feeds.
lib	Database singleton, auth, passwords, validation and media storage helpers.
prisma/schema.prisma	Database models, field types, relations and indexes.
prisma/seed.ts	Initial admin creation and sample job upserts.
scripts/check-*.mjs	Local integration checks; some create temporary data.
app/*.css	Global layout plus feature-specific styling.

How the code is kept readable

Prettier formats source consistently. Components separate presentation from server actions and storage helpers. Comments explain boundaries such as draft privacy, media cleanup and session invalidation rather than repeating every line. TypeScript checks shape compatibility; it does not replace runtime validation.

```
npm run format
npm run format:check
npm run lint
```

The script named lint currently runs `tsc --noEmit`, not a separate ESLint ruleset. Keep `package-lock.json` with the source so installs use the expected dependency versions.

04 / First local setup

1. Prepare prerequisites

Install a compatible Node.js runtime and npm. Start the MySQL service in XAMPP. Next.js runs as a Node.js process; XAMPP Apache does not serve this app and is not required for the Next.js dev server. phpMyAdmin may use Apache for its own interface.

2. Open the project terminal

Use the repository root, not a parent folder. Install the locked dependencies with `npm ci` when `package-lock.json` is present. Use `npm install` when intentionally resolving or adding dependencies.

```
npm ci
```

3. Configure environment variables

If `.env` already exists, preserve it. Otherwise copy `.env.example` to `.env` and replace placeholders. Use the actual MySQL user/password and database name. Local root without a password is not an acceptable production configuration.

```
DATABASE_URL="mysql://USER:PASSWORD@127.0.0.1:3306/di_recruitment"
AUTH_SECRET="REPLACE_WITH_A_RANDOM_SECRET_32_OR_MORE_CHARACTERS"
ADMIN_EMAIL="your-admin@example.com"
ADMIN_PASSWORD="REPLACE_WITH_A_STRONG_UNIQUE_PASSWORD"
```

Percent-encode special characters in URL credentials. Generate a random secret locally; never paste it into public documentation or commit it. Example generator:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

4. Create tables and initialize

For a new local install, apply the Prisma schema, generate the client and run the seed. Review database-change warnings; do not accept data-loss prompts on a database you need.

```
npm run db:push
npx prisma generate
npm run db:seed
npm run dev -- --port 3001
```

Visit `http://localhost:3001` and `/admin/login`. Use the initial account configured in `.env` only if it was newly created. An existing account may have a different password after an admin password change.

05 / Seed, restart & daily commands

What seed means in this project

The seed script is a starter-data script, not the database itself. It upserts four sample jobs by slug and creates an initial administrator if configured. Existing admin accounts are preserved with an empty update object, so rerunning seed does not reset their password. Sample job fields can be overwritten on rerun; do not use seed as a routine repair command.

Command	When to use it
<code>npm run dev -- --port 3001</code>	Start local development. Keep this terminal running.
<code>npm run db:push</code>	Synchronize a local schema. Review risk before applying.
<code>npx prisma generate</code>	Regenerate Client after schema/provider changes.
<code>npm run build</code>	Generate Prisma Client, compile Next.js and check types.
<code>npm start -- --port 3001</code>	Run the already-built app; production mode needs HTTPS for secure admin cookies.
<code>npm run lint</code>	Run TypeScript checks without emitting files.
<code>npm run format:check</code>	Check formatting without changing files.

Safe schema-change restart on Windows

Stop only this project's dev server with Ctrl+C. Apply reviewed schema changes, run prisma generate, then restart. An active Node process can lock Prisma's Windows engine DLL. Do not kill unrelated Node processes, erase `node_modules` blindly, or reset the database to solve a file lock.

```
npx prisma db push --skip-generate
npx prisma generate
npm run dev -- --port 3001
```

Avoid running two development servers against the same `.next` output directory. If port 3001 is occupied, identify the process and check which app owns it before stopping anything. A changed port means your old browser URL may point at another instance.

06 / Public navigation & homepage

Public page map

Route	Visitor experience
/	Hero carousel, featured card, four-job preview, services, testimonials, blog and Facebook feed.
/jobs	All active/published jobs with search, location filter and application modal.
/blog and /blog/[slug]	Published article cards and full article content.
/about/[slug]	Introduction, vision/mission/goals, leadership messages and legal documents.
/team	Published staff profiles in display order.
/gallery and /gallery/[slug]	Published event folders and their media.
/contact	Standalone enquiry form; not embedded on the homepage.

Sticky navigation

components/navbar.tsx gives the header a site-header class. CSS uses position: sticky with top: 0, keeps its original space in the layout and adds a small shadow. The small top information bar can scroll away; the main navbar stays visible. Scroll padding prevents in-page links from placing section headings directly under the navbar. Mobile menu contents can scroll if taller than the viewport.

Database-backed versus static design

Homepage carousel, jobs, blog, testimonials and the featured card use stored content. CSS and fixed marketing sections remain developer-maintained. An empty testimonial collection renders no invented endorsements. An empty published carousel uses the existing design background as a fallback.

Admin changes become visible on a new request or refresh after saving. A public tab already open does not receive real-time push updates when a different user edits content.

07 / Homepage image carousel

1. Open the editor

Sign in and choose Homepage carousel, or open `/admin/carousel`. Add a slide title/description, an image, and display order. Use a wide landscape image; the hero background uses cover cropping.

2. Upload and publish

The image must be JPG, PNG or WebP and strictly under 1 MB. Tick Show on homepage before saving if the image should be public. Admin shows Published or Draft status. Saving a draft does not make it visible to visitors.

3. Add another slide

Repeat the form for each image. Smaller `displayOrder` values appear first; IDs break ties. Editing a saved slide can replace its image. Deleting it removes the record and its owned image file.

4. Verify the public result

The carousel is the background behind the main homepage heading, not a separate gallery row. One image remains still. Two or more published slides automatically change every 5 seconds and show previous, next and pause/play controls.

Implementation details

Home reads `CarouselSlide` records filtered by `published: true`. `HeroCarousel` is a client component using `useState` for its index and `useEffect` for a five-second timer. The effect clears the timer on changes/unmount. It does not advance while the document is hidden. Navigation starts a fresh interval.

Why the default image appeared earlier

The saved slides were drafts. The public query correctly excluded them, leaving the fallback background. After publishing the two uploaded slides, both image endpoints returned HTTP 200 and the homepage contained uploaded URLs and navigation controls. Publication is separate from upload success.

Troubleshoot in this order: published checkbox saved; more than one published slide; correct localhost port; page refreshed; image request returns 200; storage files exist. A dark gradient intentionally overlays the background so the main text stays readable.

08 / Jobs & applications

1. Add a job

Open `/admin/jobs/new`. Enter title, unique lowercase slug, company, country/city, category, employment type, salary text, vacancy count, description and requirements. Optionally upload a job image under 1 MB.

2. Set visibility and priority

Published and visible maps to `Job.active`. Featured opportunity in the job form maps to `Job.featured` and changes sorting priority. This flag is separate from the independently editable homepage Featured opportunity card.

3. Use the public list

Homepage JobExplorer receives a limit of 4. It filters the published collection before slicing the visible results. See more jobs appears only when the total published collection exceeds four; it opens `/jobs` and preserves search/location filters. The separate page has no four-card limit.

4. Apply and review

A visitor selects the apply arrow and submits name, email, phone, nationality, experience and a message.

`/api/applications` creates an Application linked to the chosen Job. Admin reviews records at

`/admin/applications` and changes their status.

Status	Meaning
RECEIVED	New application waiting for review.
REVIEWING	Under assessment.
SHORTLISTED	Candidate selected for further consideration.
REJECTED	Not proceeding.
PLACED	Recorded as successfully placed.

Deletion warning: the schema cascades Job deletion to its related Application records. Prefer hiding/closing a job when its application history must be retained. Back up before permanent deletion.

The current application UI does not upload CV files, even though Application has a nullable `cvUrl` field. A field in the schema does not mean the complete user workflow has been implemented.

09 / Blog & About content

Blog workflow

1. Open /admin/blog/new. 2. Enter a title; the form suggests a lowercase hyphenated slug until manually changed. 3. Write an excerpt and full article. 4. Optionally upload an image under 1 MB. 5. Tick Publish on website and save. Draft posts stay out of public listings and public article requests return not found.

Edit existing posts from /admin/blog. Uploaded images appear on cards and the article page. The editor supports replacement/removal. Duplicate slugs return a friendly error. When a slug changes, affected routes are revalidated. Deleting a post removes its owned image.

About menu and editors

Public slug	Managed content
introduction	Company introduction.
vision-mission-goals	Vision, mission and goals.
chairman-message	Chairman message and optional portrait.
managing-director-message	Managing Director message and optional portrait.
legal-documents	Introductory text plus separately managed document images.

Open /admin/pages, choose a page, change title and text, then save. The two leadership pages expose the image uploader with preview, replace and remove options. Text is rendered as text with preserved paragraph breaks rather than arbitrary HTML.

lib/content-pages.ts provides fallback content when no saved ContentPage record exists. About edits publish immediately; these pages do not have the same draft checkbox as blog posts. Leadership image uploads become publicly accessible with their saved pages.

Formatting content

Use blank lines between paragraphs. This is not a rich-text editor: entering HTML does not create custom page markup. Developer edits to layout belong in the page/component/CSS files, not in database text fields.

10 / Team & legal documents

Team profiles

Open `/admin/team`, add a member and enter their name, position, biography and display order. Upload a portrait directly instead of typing a photo URL. Publish the profile when ready. The public `/team` page reads published members ordered by `displayOrder` and `ID`; job position is editable text, not an authorization role.

Portraits accept JPG, PNG or WebP below 1 MB. Sharp normalizes orientation, resizes within 800 x 1000 pixels without enlarging, strips metadata through re-encoding, and saves WebP. Admin can replace or remove a photo. A hidden member's local portrait requires a signed-in admin to view it.

Legal documents

1. Create an entry

Open `/admin/documents`. Enter a document title and display order, then upload a scanned document image under 1 MB.

2. Publish

Tick Publish document and save. The public Legal Documents page shows the image thumbnail and title. Clicking opens the full image in a new tab.

3. Replace or delete

Choose a new image in the same entry to replace the old file. Delete document removes the entry and its owned local file. There is no recycle bin.

The previous public-URL input has been replaced by upload. Existing external links remain readable until replaced; the storage cleanup never attempts to delete a remote URL. The current uploader accepts images, not PDFs or Word files.

Privacy before upload

Only publish business documents appropriate for public access. Redact private identification, signatures or account information when necessary. The app does not automatically redact a document. The operator is responsible for reviewing its contents and obtaining permission for staff portraits.

11 / Event gallery & Facebook

Folder-based gallery, step by step

1. Open /admin/gallery and create a folder/album. 2. Add its title, slug, description, event date and order. 3. Upload media inside that folder. The UI accepts up to 20 selected files per batch, while the upload endpoint handles an individual file request. 4. Edit captions and display order. 5. Publish the album when all media is ready.

The first photo by display order is used as the folder cover. Public visitors browse /gallery and a folder-specific /gallery/[slug] page. Draft albums and their media are restricted. Deleting an album removes related media records and owned files; deleting one media item does not delete the album.

Media	Accepted formats and strict limit
Photo	JPG, PNG, WebP; less than 1,048,576 bytes.
Video	MP4, WebM; less than 15,728,640 bytes.
Transport	Gallery endpoint caps the request body as it is read; allow multipart overhead in your proxy configuration.

Gallery validates recognized file signatures and stores the original media bytes. It does not perform the same full image decode/re-encode pipeline as content/team photos, and it is not a full video codec verification or malware scanning system. Video serving supports byte-range requests for seeking.

Facebook feed

The homepage and admin overview embed a public Facebook Page timeline. FACEBOOK_PAGE_URL selects the source; restart after changing environment configuration. Refresh feed reloads the embed. Posts are loaded by Facebook, not imported into MySQL.

This is not a personal-account login, Facebook posting tool, or guaranteed real-time synchronization. Privacy settings, restricted pages, browser tracking protection and Facebook sign-in requirements can prevent loading. A direct Page link remains available.

12 / Contact enquiries

Visitor experience

Contact links open /contact, a separate page. The homepage only shows contact information and a button to reach it. The form collects full name, email, phone, subject and message. On success it displays a confirmation and a link back to the homepage.

1. Validate

The server trims input and validates a name of 2-100 characters, email up to 254 characters, phone 7-30 characters, subject 3-180 characters and message 10-5,000 characters. Phone characters are restricted to digits and common telephone punctuation.

2. Reduce basic spam

A hidden website field is a honeypot. The server also counts recent submissions for the normalized email and allows fewer than three existing messages in a 15-minute window. This is basic mitigation, not a comprehensive bot-defense system.

3. Store privately

Prisma creates ContactSubmission. No public submission-list endpoint exists. The success message does not email the visitor or the admin; notifications have not been implemented.

4. Review in admin

Open /admin/contacts. Messages appear newest first with subject, person, email, phone, full message and timestamp. The list has pages of 20 records and previous/next links.

5. Delete carefully

Click Delete submission and confirm. The action rechecks administrator authentication and validates the ID before deleting. Deletion is permanent in the app; recovery requires a suitable backup.

Contact versus application

A contact message is a general enquiry and has no job relation or hiring status. Applications are a separate hiring workflow. The two are intentionally stored in different models and have different admin screens.

Before public launch, establish a retention policy and access policy for personal information. Add distributed rate limiting, monitoring and abuse controls appropriate to expected traffic; the current email-count check can race under simultaneous requests.

13 / Testimonials

1. Add genuine feedback

Open `/admin/testimonials`. Enter the person's name, role/location, quotation and display order. Obtain permission before publishing a named testimonial. No default endorsement is inserted when the collection is empty.

2. Publish or keep a draft

Tick Publish on homepage and save. Drafts remain in admin. Edit a record to change its text or order. Delete testimonial asks for confirmation and permanently removes it.

3. Check the carousel

With one published item, a single card stays static. With more than one, the carousel changes every 5 seconds. Visitors can navigate previous/next or pause/play. Only published entries are passed to the public client component.

Card design

Cards use a light surface, rounded corners, a green-to-gold top accent, quote icon, readable quotation, a circular initials avatar and separate name/role text. These avatars are initials, not uploaded photos. The section is responsive and constrains its width for comfortable reading.

Implementation

The server component `components/testimonials.tsx` fetches ordered records. `components/testimonial-carousel.tsx` owns client state and the interval. A flex track moves with `translateX` and a 600 ms CSS transition. Each slide takes the viewport width; overflow is hidden. Reduced-motion preference disables the CSS transition, and pause controls remain available.

The interval is cleaned up when the component unmounts or dependencies change. Hidden browser documents do not advance on a tick. Non-current slides are marked `aria-hidden` so assistive technology is not asked to read every offscreen quotation at once.

Verification checklist

Try zero, one and two published testimonials. Check that a draft never appears publicly, a single item has no unnecessary carousel controls, two items cycle and wrap, pause holds the current item, and long quotations remain legible on a narrow screen.

14 / Featured opportunity card

The original homepage card contained fixed Greece recruitment copy. It has been replaced by an admin-controlled card. Open `/admin/featured-opportunity` to create, edit, hide or delete it.

Field	Purpose
Title	Heading shown inside the hero card; 3-180 characters.
Description	Short opportunity summary; 10-500 characters.
Badge	Short label such as GR; 1-8 characters.
Location	Destination/location text; 2-100 characters.
Openings	Integer from 1 to 100,000.
Show on homepage	Whether the saved card is visible to visitors.

1. Add

Complete the fields and save. New entries default to showing on the homepage unless unchecked.

2. Edit

Return to the same screen and update the existing card. This editor manages one homepage highlight, not a collection of rotating featured cards.

3. Hide or delete

Uncheck Show on homepage to retain a draft. Delete removes the card only; ordinary jobs and their applications are not deleted.

Storage choice

`lib/featured-opportunity.ts` defines a Zod schema for the structured card. A `ContentPage` record with slug `homepage-featured-opportunity` stores its fields as JSON inside content. The public helper parses and validates that JSON before rendering. Invalid or missing content does not resurrect the old static card.

The View opportunities link opens `/jobs`. It does not point to an individual job and there is no custom destination URL field. This card is distinct from `Job.featured`, which only prioritizes job listings. Update both independently if you want them to describe the same hiring campaign.

15 / Authentication & security

Implemented controls

Admin passwords are hashed with scrypt and a random 16-byte salt; the stored string contains salt and hash, not the plaintext password. Verification rejects malformed records and oversized input, and uses a timing-safe comparison. Auth helpers are in `lib/passwords.ts` and `lib/auth.ts`.

A signed session cookie contains identity, expiry and a password-version value. HMAC-SHA256 signs the payload using `AUTH_SECRET`. The cookie is HTTP-only, SameSite Strict, valid for eight hours and Secure in production. Session checks also look up an active admin in the database.

Password change requires the current password and a new password of 12-128 characters, matching confirmation and a different value. The update is guarded against a simultaneous change. A new cookie is issued; the password-version check invalidates older cookies. The password-change action has an in-memory five-attempt/15-minute limiter.

Important limitations, not guarantees

The login action currently has no explicit login-attempt throttle, MFA or forgot-password email workflow. The password-change limiter is process-local, not shared across servers. Contact spam protection is basic. The public application endpoint lacks comprehensive abuse limiting and should explicitly reject applications for inactive/expired jobs before launch.

Server actions recheck authentication for admin mutations. Gallery upload additionally checks Origin. These are useful controls, not proof that the entire app has passed penetration testing. Escaped React text avoids rendering user content as raw HTML, but all new features still need validation and authorization reviews.

Production priorities

Use HTTPS, a non-root database account, restricted database networking, secret management, login throttling, secure backup access, monitored logs and a retention policy. Add administrator roles/audit trails if multiple staff need different permissions. Never expose `.env`, password hashes or personal submissions in debug output.

16 / Upload lifecycle

1. Choose and preview

The browser validates image size and creates a temporary object URL for a preview. This is convenience only; a malicious client can bypass it. Server validation is authoritative.

2. Validate on the server

Content/team helpers check size and recognized image signatures. Sharp decodes pixels with a 20-million-pixel input cap, normalizes orientation, resizes and re-encodes to WebP. Content images fit within 2000 x 2400; team portraits fit within 800 x 1000. Output size is checked again.

3. Assign a safe key

A random UUID filename is generated. The storage helper accepts only the expected key format and writes outside public/. It never uses an arbitrary uploaded path or remote URL as a deletion target.

4. Commit database references

The action stores a URL/reference after successful validation. If saving fails, it attempts to remove the newly stored file. If a replacement succeeds, it removes the previous owned image. Some actions use updatedAt checks to reject stale concurrent edits.

5. Serve according to visibility

Content image routes resolve the owning post, job, document, page or carousel slide. Draft records require admin access. About page images are public because the page itself has no draft state. Responses use image/webp, nosniff and private/no-store headers.

Storage setting	Default directory
GALLERY_STORAGE_DIR	storage/gallery
TEAM_STORAGE_DIR	storage/team
CONTENT_IMAGE_STORAGE_DIR	storage/content

Strictly under 1 MB means exactly 1,048,576 bytes is rejected; exactly 15,728,640 bytes is rejected for video. The code uses binary byte thresholds despite the interface saying MB. Server actions have a 2 MB request limit; gallery uploads use a separate route for larger videos.

Files and database changes are not a single filesystem/database transaction. Cleanup reduces orphan files but cannot eliminate every crash scenario. Back up both sources and periodically reconcile missing/orphaned files before deletion.

17 / Database reference

Model	Stored information / key relationship
Admin	Unique email, name, script hash, active flag and timestamps.
Job	Job details, unique slug, image URL, vacancies, active/featured flags; parent of Application.
Application	Candidate contact and experience, message, status, jobId; job deletion cascades here.
BlogPost	Unique slug, title, excerpt, long article, image URL and publication state.
TeamMember	Name, position, biography, portrait, ordering and publication.
ContentPage	Slug primary key; title, long content, optional image. Also stores the single featured card as JSON.
LegalDocument	Title, local image URL or legacy external URL, order and publication.
GalleryAlbum	Unique slug, description, event date, order and publication.
GalleryMedia	Album relation, unique storage key, original name, MIME, size, caption and order.
CarouselSlide	Hero image URL, title, order, published and updatedAt.
Testimonial	Name, role/location, quote, order, published and updatedAt.
ContactSubmission	Visitor contact details, subject, message and createdAt.

GalleryMedia belongs to GalleryAlbum with cascade deletion. Most other content collections are independent. Keep these distinctions when migrating or deleting records. ContentPage has no integer ID and therefore needs no sequence reset during PostgreSQL import.

Indexes support publication/order lookups, application job/email searches and contact chronology. The checked project has schema.prisma and seed.ts but no committed Prisma migration history. Its local setup currently uses db push; production migration discipline must be introduced deliberately.

18 / Tests & acceptance checks

Existing verification commands

```
npm run lint
npm run format:check
npm run build
node scripts/check-local.mjs
node scripts/check-content.mjs
node scripts/check-gallery.mjs
node scripts/check-account-team.mjs
node scripts/check-content-images.mjs
```

Run integration scripts only against a local/staging instance with known credentials and a backup. They are not read-only: they create temporary records/files and normally clean them up. Some scripts use ADMIN_EMAIL/ADMIN_PASSWORD; those values may be stale after changing the real admin password. Other scripts create temporary accounts.

What prior checks established

During implementation, production builds and TypeScript checks passed at several checkpoints. Image integration checks exercised valid/invalid uploads, the size boundary, draft privacy, publication, replacement, removal and leadership image display. HTTP checks confirmed the homepage job limit and the full jobs list. A later homepage/admin-route check covered the featured-card change. These results are historical checkpoints, not a claim of complete continuous coverage of every latest interaction.

Manual acceptance before launch

1. Sign in/out and change password; verify old sessions fail. 2. Create, edit, publish and hide each content type. 3. Test four and five jobs: the button must appear only above four. 4. Submit a real-format test application and contact enquiry, inspect them in admin, then remove only test data. 5. Test carousels for 0/1/2 items and pause/navigation. 6. Test mobile navigation and long content. 7. Replace/delete uploads and verify old URLs no longer expose them.

Also test anonymous access to admin routes, direct draft image URLs, invalid IDs, duplicate slugs, database outages and upload limits. PostgreSQL rehearsal must repeat this matrix rather than assuming a successful schema push means the migration is complete.

19 / Troubleshooting guide

Symptom	Likely cause and next check
Cannot read properties of undefined (findMany)	Generated Prisma Client predates a model. Stop this app, apply schema, generate Client and restart. Do not hide the error by returning fake data.
Prisma EPERM / DLL rename	Windows engine file is locked by a project Node process. Stop verified dev/build processes, then regenerate.
Database connection failure	Check XAMPP MySQL, host/port, credentials and database name. Opening the XAMPP panel is not enough if MySQL is stopped.
Carousel shows default	No published slides, wrong server port, stale page or missing stored file. Confirm saved Published status.
Only one image / no movement	Only one published record, carousel paused, or tab hidden. Publish at least two and wait five seconds.
No See more jobs button	Four or fewer published jobs. The button intentionally appears only above four.
Blog or team missing	Draft/hidden state, missing record or wrong route. Refresh after saving.
Image rejection	Size at/above limit, unsupported type, decode failure, pixel cap or generated output still too large.
Admin logged out	Cookie expired, password changed, secret changed or account disabled. Sign in again.
Facebook blank	External embed restrictions, privacy settings or Page visibility; use the direct link.

Never solve a display problem by running migrate reset, deleting the database or reseeding indiscriminately. Find whether the failure is data visibility, database connectivity, generated code, media storage or browser state first.

20 / Deployment & maintenance

1. Prepare a staging environment

Use a Node.js-capable server with a persistent writable volume. Plain static hosting cannot run Prisma, server actions, uploads or sessions. Keep staging database and media separate from production.

2. Set secrets and storage

Configure DATABASE_URL, AUTH_SECRET and desired Facebook Page. Mount all three upload folders or set absolute storage environment paths. Use restricted permissions and private database access. Never deploy the example credentials.

3. Manage schema deliberately

The current project used db push locally. Establish a reviewed migration history before production changes; baseline an existing populated database instead of resetting it. Use a separate staging database to rehearse structural changes.

4. Build and start

Run npm ci and npm run build, then npm start under a managed process/service. Serve through HTTPS. Configure reverse proxy request limits for the gallery video maximum plus overhead, preserve host/origin correctly, and verify admin cookies work.

5. Back up and restore-test

Back up database, storage/gallery, storage/team and storage/content together. Protect copies of required environment configuration separately. Verify restoration into a temporary environment; a backup that has never been restored is unproven.

6. Monitor and maintain

Monitor HTTP failures, database health, disk capacity and failed uploads. Review dependency/security updates in staging. Do not combine a major framework/ORM upgrade with a database-engine migration unless separately tested.

Known operational boundaries

This project has not been publicly deployed as part of this documentation task. It has no automated email notifications, background media scanning, shared distributed limiter, audit log or trash/recovery UI. Large data volumes will need server-side pagination for more lists; /jobs currently loads the complete active collection for browser filtering.

Keep an encrypted, access-controlled backup of user data. Deleting a contact, job or file is a business action with consequences. A job deletion also deletes related applications. Document who is allowed to perform destructive operations.

21 / Sources & verification scope

Implementation details were checked against the workspace source, including schema.prisma, package.json, README.md, public/admin routes, forms, auth helpers, storage helpers and verification scripts. This handbook does not reproduce private .env values or customer data.

External references for the final migration section

Use the Prisma 6 documentation for this project. Later Prisma major versions can use different configuration and migration commands. References were checked on 23 September 2026.

1. Prisma 6 connection URLs

<https://docs.prisma.io/docs/orm/v6/reference/connection-urls>

2. Prisma PostgreSQL connector and native types

<https://docs.prisma.io/docs/orm/core-concepts/supported-databases/postgresql>

3. Prisma 6 development / production migrations

<https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>

4. Prisma 6 getting started with migrations

<https://www.prisma.io/docs/orm/v6/prisma-migrate/getting-started>

5. Prisma 6 case sensitivity

<https://www.prisma.io/docs/orm/v6/prisma-client/queries/case-sensitivity>

6. PostgreSQL sequence functions

<https://www.postgresql.org/docs/16/functions-sequence.html>

7. PostgreSQL pg_dump

<https://www.postgresql.org/docs/current/app-pgdump.html>

Important distinction

The following PostgreSQL instructions are a project-specific migration plan, not an executed conversion. No production cutover, data export, new PostgreSQL installation or database deletion was performed to create this PDF. Rehearse with a copy, validate data and obtain a maintenance window before applying it to a live site.

22 / PostgreSQL: plan & protect

Changing DATABASE_URL alone is not enough. The provider, MySQL-only native types, generated Client, schema history, existing rows, stored files and identity sequences must all be considered. Keep Prisma 6 for this change rather than upgrading the ORM at the same time.

1. Choose the migration track

For a new empty install with no needed content, create an empty PostgreSQL database, convert the schema, initialize it and seed once. For the current populated website, use the data-preserving track below. Never seed the target before importing existing IDs and records.

2. Create a recovery point

Copy the source code and lockfile into a separate migration workspace or branch. Keep the original MySQL schema and environment configuration. Back up MySQL using XAMPP mysqldump/phpMyAdmin and separately copy all media directories. Use an actual restore test.

```
& "C:/xampp/mysql/bin/mysqldump.exe" -u USER -p `
  --single-transaction --routines --triggers `
  --result-file="D:/backups/di-recruitment-before-pg.sql" `
  di_recruitment
```

Create the backup directory first. The password is prompted by -p; do not put it into shell history. A MySQL SQL dump is for restoring MySQL, not something to execute directly in PostgreSQL. For consistent copies, pause writes during the final backup and migration. Include file changes in that freeze.

3. Inventory data

Record row counts and IDs for every model on page 19. Note database version, charset/collation and timestamp assumptions. Check duplicate emails/slugs under your intended PostgreSQL case rules. Verify each local media reference has a matching file.

4. Prepare PostgreSQL

Install a supported PostgreSQL release or provision a managed instance. Create an empty UTF-8 database owned by a dedicated app role. Use a separate migration role if needed for DDL. In a local GUI such as pgAdmin, create the login and database without placing a password in shared scripts.

Name the staging database distinctly, for example di_recruitment_pg_stage. Do not overwrite or remove the MySQL database. XAMPP MySQL and PostgreSQL are separate services; PostgreSQL commonly listens on port 5432.

23 / PostgreSQL: schema conversion

5. Change the provider in the migration copy

In `prisma/schema.prisma`, change only the `datasource` provider first. Keep the Prisma 6 `datasource` URL convention used by this app. [1,2]

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

6. Set the target connection URL

Use credentials for the new target, not the source MySQL user. Percent-encode reserved password characters. A hosted instance may require additional TLS options supplied by the provider. [1]

```
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/di_recruitment_pg_stage?schema=public"
```

7. Replace incompatible native annotations

The current `ContentPage.content` and `BlogPost.content` use `@db.LongText`, which is MySQL-specific. Change both to `@db.Text`. The project's `@db.VarChar(n)`, `@db.Text` and `@db.Date` fields can remain. Keep the application enum, relations and autoincrement model declarations, then validate the generated PostgreSQL schema. [2]

```
// ContentPage
content String @db.Text

// BlogPost
content String @db.Text
```

8. Validate before creating tables

Stop the migration copy's app process, run Prisma `format/validate` and check any errors. No data is moved by generating the Client.

```
npx prisma format
npx prisma validate
npx prisma generate
```

Do not miss application-level text

`components/application-modal.tsx` currently mentions MySQL in an error message. Change database-specific UI wording to a generic submission failure during the migration. Review README and environment examples too. Public routes and Prisma method names generally remain the same after successful regeneration.

Do not change the active MySQL workspace until you have a tested PostgreSQL copy. The safest migration rehearsal runs on a different port and has its own `.next` output, dependencies and environment.

24 / PostgreSQL: initialize & transfer

9. Create a target-specific migration history

This checked project has no prisma/migrations directory. On an EMPTY PostgreSQL staging database, create an initial Prisma 6 migration. If a future project copy already has MySQL migrations, preserve/archive that history separately; do not replay MySQL SQL as PostgreSQL SQL. [3,4]

```
npx prisma migrate dev --name init_postgresql
npx prisma generate
```

migrate dev can require a shadow database or permission to create one. If permissions are restricted, configure a distinct disposable shadow database; never point it at your source or target data database. For a disposable local prototype only, db push is an alternative to establishing history, not a data transfer tool.

10. Export rows from the original MySQL copy

Keep the original MySQL-generated Client in that source workspace. Write a reviewed one-off export script using findMany for all 12 models. Export scalar fields only, preserving IDs, foreign keys, timestamps, nulls, flags and exact strings. Store JSON in a private backup directory, not public/ or source control.

11. Import with the PostgreSQL-generated Client

Use a separate import script in the converted workspace. Parse and validate the export. Restore DateTime and Date fields from ISO strings to Date objects as needed. Insert into empty target tables in batches, retaining source IDs. Abort on duplicates rather than silently skip them. Do not merge into an unknown populated target.

Order	Models
Independent / parent rows	Admin, Job, GalleryAlbum, ContentPage, LegalDocument, TeamMember, BlogPost, CarouselSlide, Testimonial, ContactSubmission
Child rows after parents	Application after Job; GalleryMedia after GalleryAlbum

Transfer Admin.passwordHash exactly; never re-hash an existing hash. Copy ContentPage.content unchanged, including featured-opportunity JSON. Preserve Application.status enum values. Gallery storageKey and all URL strings must keep matching files.

This describes the transfer contract; the project does not yet contain a production-ready MySQL-to-PostgreSQL export/import tool. Build and rehearse that utility before touching live data. A raw MySQL dump cannot safely substitute for this typed transfer.

25 / PostgreSQL: IDs, files & behavior

12. Reset integer sequences after import

Explicit imported IDs may not advance PostgreSQL sequences. Without resetting them, a later admin create can collide with an existing ID. Run the following example for Job, then repeat for every integer-ID model listed below. [6]

```
SELECT setval(
  pg_get_serial_sequence('Job', 'id'),
  COALESCE((SELECT MAX(id) FROM "Job"), 1),
  EXISTS (SELECT 1 FROM "Job")
);
```

Repeat for Admin, Application, GalleryAlbum, GalleryMedia, LegalDocument, TeamMember, BlogPost, CarouselSlide, Testimonial and ContactSubmission. Preserve quoted mixed-case table names unless you deliberately mapped different names. ContentPage uses a string slug key and has no ID sequence. The third argument lets an empty table issue its first ID correctly.

13. Copy media without rewriting URLs

Move storage/gallery, storage/team and storage/content to persistent target paths, preserving filenames. Configure the three storage environment variables. The /api/... URLs can remain unchanged when the app retains its routes. Database import does not transfer any file bytes.

14. Review case sensitivity

The homepage/all-jobs client filter lowercases strings, so that browser behavior is independent of SQL collation. However, app/api/jobs/route.ts uses Prisma contains filters. PostgreSQL filtering may be case-sensitive where MySQL collation previously was not. Add mode: "insensitive" to appropriate PostgreSQL string filters if equivalent behavior is wanted. [5]

```
// PostgreSQL Prisma Client filter example
{ title: { contains: q, mode: "insensitive" } }
```

Apply the same decision to company/category search. Do not assume this makes unique constraints case-insensitive: slugs/emails and uniqueness require their own normalization/data review. Preserve current email normalization and resolve any conflicting records before import.

15. Check date and text fidelity

Compare timestamps, date-only event dates, Unicode/Nepali text, multiline messages, salary symbols and article lengths. MySQL timezone and collation assumptions can differ. Verify representative values, not just successful row insertion.

26 / PostgreSQL: rehearsal & cutover

16. Validate the imported copy

Compare counts for all 12 models; confirm IDs and parent-child references. Check selected record fields and media hashes/availability. Create a temporary record in each integer-ID table to verify sequences, then remove only those test records. Check admin password verification without exposing stored hashes.

17. Run the complete application

Run `npm run lint` and `npm run build`. Start the staging copy on a distinct port. Exercise sign-in, password change, content CRUD, draft privacy, image replacement, gallery playback, job applications, contact deletion and both carousels. Confirm the homepage limit is still four and `/jobs` includes every active job.

18. Prepare production schema

Use the reviewed PostgreSQL migration files to create an empty production target with Prisma 6 migrate deploy. That command applies schema migrations, not user-data conversion; use the rehearsed import process separately. [3]

```
npx prisma migrate deploy
npx prisma generate
npm run build
```

19. Freeze and perform final transfer

Announce maintenance; block public applications/contact writes and admin edits/uploads. Stop background writers if any have been added. Make fresh MySQL/media backups, export the final dataset, import into the clean target, repair sequences and synchronize media again.

20. Switch and verify

Deploy the PostgreSQL-built app with its target `DATABASE_URL` and persistent media configuration. Rotate `AUTH_SECRET` if you intentionally want all admins signed out. Otherwise preserve it securely with the imported hashes. Restart all app processes so no stale MySQL Client survives. Run smoke checks before reopening writes.

Go/no-go: do not reopen if counts differ unexpectedly, media is missing, authentication fails, new records collide, or submissions cannot be created/reviewed. Keep the original MySQL database and application release intact until the observation period and backups are accepted.

27 / PostgreSQL: rollback & finish

Rollback before reopening writes

If verification fails while maintenance mode is still active, switch back to the preserved MySQL application build and its environment configuration. Restore the matching media snapshot only if needed and validated. Restart the app and verify source data. A PostgreSQL-generated Client cannot be repaired just by replacing its URL with a MySQL URL; deploy the original build or regenerate from the original MySQL schema.

Rollback after accepting new submissions

Do not simply point back to the old database: new PostgreSQL applications, contacts, edits and uploads would be lost. Freeze writes again, identify and reconcile the new/changed records and files, then execute a reviewed reverse-transfer or choose to repair PostgreSQL in place. This is why the first verification window should happen before accepting public writes.

PostgreSQL backup example

Use `pg_dump` for a PostgreSQL database export and `pg_restore` for its custom archive format. Keep media backups separately; database archives do not include the app's upload folders. Restore to a fresh test database and inspect warnings. [7]

```
pg_dump -h localhost -U APP_USER -d di_recruitment_pg `
-Fc -f "D:/backups/di-recruitment-pg.dump"
```

```
pg_restore -h localhost -U RESTORE_USER `
-d di_recruitment_restore_test --no-owner `
"D:/backups/di-recruitment-pg.dump"
```

Final migration acceptance checklist

Database counts and relationships match. All images/videos are present. Admin sign-in and session invalidation work. Add/edit/delete operations succeed. Published/draft visibility is unchanged. Both carousels rotate at five seconds. Contact remains on its own page. Four jobs appear on the homepage and additional jobs are linked. Backups restore successfully. Monitoring and rollback ownership are documented.

Result: the frontend, admin workflows and Next.js routes can stay substantially the same; Prisma's provider-specific schema/client and the storage/database deployment become the PostgreSQL-backed implementation. Finish only after the data-preserving checks pass, not merely when the homepage loads.

End of handbook

Current app: MySQL. PostgreSQL switch: planned, not executed.